

Mock Movie 데이터 · ListView · ListView.separated · GridView

1. Mock Movie 데이터 — 세 화면이 같은 데이터를 읽게 하는 것

```
class Movie {
  const Movie({
    // const 생성자 = 불변 객체
    required this.id, // 이름 있는 필수 파라미터
    required this.title, required this.genre,
    required this.year, required this.posterAsset,
  });
  final int id; // 상세 Route의 Path Parameter
  final String title; final String genre; final int year;
  final String posterAsset; // assets/images/... 경로
}

const movies = [
  Movie(id: 1, title: '별빛 아래 우리', genre: '드라마',
    year: 2024, posterAsset:
    'assets/images/movie_1.png'),
  Movie(id: 2, title: '우주의 끝에서', genre: 'SF', ...),
];

Movie? findMovieById(int? id) { // 반환 타입이 nullable
  for (final movie in movies) {
    if (movie.id == id) return movie;
  }
  return null; // 못 찾으면 null
} // ↓ 상세 화면: findMovieById(movieId) ?? 404 화면
```

왜 한곳에 모으냐

홈·목록·상세가 각자 하드코딩하면 같은 '별빛 아래 우리'가 화면마다 다르게 보인다. 위크북 요구사항: 세 화면이 동일한 ID와 제목을 쓸 것.

const 생성자

값이 변하지 않는 객체. 컴파일 타임에 하나만 만들어 재사용 → 리빌드해도 새로 안 만든다

required

이름 있는 파라미터를 필수로. 빠뜨리면 컴파일 에러

findMovieById가 Movie? 를 돌려주는 이유

URL을 직접 치거나 (/movies/999) 잘못된 ID로 들어올 수 있다. null을 돌려주면 상세 화면이 "영화를 찾을 수 없어요"를 그릴 수 있지만, !로 강제하면 앱이 죽는다.

Extra가 아니라 ID로 다시 찾는 이유

extra: movie 는 URL에 안 남아 앱 재시작·URL 직접 접근·Deep Link에서 null이 된다. 그래서 Path Parameter의 ID → findMovieById 가 기본.

내 구현: lib/data/mock_movies.dart — 영화 6편 + genres (중복 없이 뽑아 Chip에 사용) + findMovieById. movies 가 const 라 정렬할 땐 [...movies] 로 복사본을 만든다.

2. ListView — 한 방향으로 나열하고 스크롤

```
ListView.builder(
  itemCount: items.length, // 항목 개수
  itemBuilder: (context, index) { // index마다 호출
    final item = items[index]; // List에서 하나 꺼내기
    return /* item으로 Widget 생성 */;
  },
) // scrollDirection, padding은 선택
```

itemCount

생성할 항목 개수

itemBuilder

각 항목 Widget 생성. 보이는 것만 만든다(1000개여도 가법다)

ListView.separated — 항목 사이에 간격·구분선

```
ListView.separated(
  itemCount: items.length,
  separatorBuilder: (context, index) => const
  SizedBox(width: 8),
  itemBuilder: (context, index) => /* Widget */,
)
```

separatorBuilder 는 항목 사이에만 호출된다(3개면 2번). 마지막 뒤에 굳 더거 여백이 안 붙는다. 세로는 Divider(), 가로는 SizedBox(width:) .

가로 ListView는 높이를 스스로 못 정한다

세로 목록은 부모가 높이를 주지만, 가로 목록은 높이를 정해 주는 부모가 없으면 에러가 난다. SizedBox(height: ...) 로 감싼다.

```
SizedBox(
  height: 236, // ← 없으면 터진다
  child: ListView.separated(
    scrollDirection: Axis.horizontal,
    padding: const EdgeInsets.symmetric(horizontal: 16),
    itemCount: movies.length,
    separatorBuilder: (c, i) => const SizedBox(width: 12),
    itemBuilder: (context, index) {
      final movie = movies[index];
      return MovieCard(movie: movie, width: 136,
        onTap: () => onMovieTap(movie));
    },
  ),
)
```

홈의 "지금 인기 있는 영화" 줄이 이 구조다. 가로 목록은 셀 너비도 직접 정한다 (width: 136) .

3. GridView — 행과 열의 격자

```
GridView.builder(
  itemCount: items.length, // ListView와 동일
  itemBuilder: (context, index) => /* Widget */,
  gridDelegate: // ← 차이점: 배치 방식
    const SliverGridDelegateWithFixedCrossAxisCount(
      crossAxisCount: 2, // 한 줄에 2개
      crossAxisSpacing: 12, // 좌우 간격
      mainAxisSpacing: 16, // 위아래 간격
      childAspectRatio: 0.65, // 가로 / 세로
    ),
)
```

itemCount · itemBuilder 는 ListView.builder와 같고,

gridDelegate 로 배치 방식을 추가로 정하는 것만 다르다.

childAspectRatio 감 잡기 — 셀의 가로 ÷ 세로. 작을수록 세로로 길어진다. 포스터 2:3(≈0.67)에 제목·장르 두 줄이 더 붙으므로 그보다 작게. 내 목록은 0.6.

셋 중 무엇을 쓸까 — MovieLog 적용

Widget	쓰는 경우	내 화면
ListView.builder	한 방향으로 나열	—
ListView.separated	항목 사이에 간격·구분선	홈 가로 줄(12px), 장르 Chip 줄(8px)
GridView.builder	한 줄에 2개 이상, 여러 열	영화 목록 2열

함정 — Column 안에 ListView 를 그냥 넣으면 "unbounded height" 에러. Expanded 로 감싸거나 SizedBox(height:) 로 고정.

커넥션 풀 (Connection Pool)

1. 문제 — DB 연결은 비싸다

API가 호출될 때마다 서버가 MySQL의 문을 두드리고 비밀번호를 확인하며 연결을 맺는다면, 서버는 **연결을 맺고 끊는 데에만 시간을 쓰다가 뻥는다**.

[매 요청마다]

요청 → TCP 3-way → 인증 → 세션 준비 → 쿼리(1ms) → 종료
 요청 → TCP 3-way → 인증 → 세션 준비 → 쿼리(1ms) → 종료
 └─ 준비에 수십 ms, 정작 쿼리는 1ms

연결 한 번에 드는 것: TCP 3-way 핸드셰이크 + 계정 인증 + 세션 변수·문자셋 설정. 쿼리 자체보다 **준비 과정이 훨씬 비싸다**.

2. 해결 — 미리 뚫어두고 빌려 쓴다

[커넥션 풀]

서버 시작 → 핫라인 10개를 바구니(Pool)에 미리 꽂아둠

요청 → [풀에서 1개 빌림] → 쿼리 → [반납]
 요청 → [풀에서 1개 빌림] → 쿼리 → [반납]
 전부 나가 있으면 → 대기(큐)

connectionLimit 미리 뚫어둘 커넥션 개수. 보통 10

maximumPoolSize

waitForConnections 풀이 꽉 찼을 때 여러 대신 대기할지

queueLimit 대기줄 최대 길이(0 = 무제한)

한 줄 정의

커넥션 풀 = 비싼 DB 연결을 미리 여러 개 만들어 두고 요청마다 빌려주고 돌려받는 바구니.

4. 내가 직접 확인한 것 — 풀은 첫 쿼리 때 만들어진다

서버 기동 로그에는 Hikari가 없어서 DB 연결이 안 된 줄 알았는데, GET /books 를 처음 호출하자 그때 찍혔다.

```
Tomcat started on port 8080
Started StudyApplication in 1.065 seconds
  ↳ 여기까지 Hikari 없음 (요청 0건)
HikariPool-1 - Starting...
HikariPool-1 - Added connection com.mysql...
HikariPool-1 - Start completed.
  ↳ GET /books 첫 호출 시점
```

HikariCP는 **lazy 초기화**다. 기동 시점에 DB 접속을 검증하고 싶으면

`spring.datasource.hikari.initialization-fail-timeout` 을 양수로 두거나, 시작 시 `SELECT 1` 을 날리는 `CommandLineRunner` 를 둔다.

6. 예상 질문 (스터디·면접)

- Q. 커넥션 풀을 한 문장으로? 비싼 DB 연결을 미리 N개 만들어 두고 요청마다 빌려주고 돌려받아, 연결 생성 비용을 없애는 구조.
- Q. 풀이 없으면 구체적으로 뭐가 문제인가? 요청마다 TCP 핸드셰이크·인증·세션 준비가 붙어 실제 쿼리(1ms)보다 준비(수십 ms)가 커진다. 트래픽이 몰리면 연결 작업만 하다가 서버가 뻥는다.
- Q. Spring은 왜 풀 설정 코드가 없나? Spring Boot가 `application.yml` 의 `datasource` 정보를 읽어 HikariCP와 JdbcTemplate을 자동으로 Bean 등록하기 때문. NestJS는 순수 드라이버를 쓸 때 Provider로 직접 조립해야 한다.
- Q. 풀에 남은 커넥션이 없으면? `waitForConnections: true` 면 대기줄에 서고, 타임아웃이 지나면 예외. `false` 면 즉시 에러.
- Q. 커넥션 누수는 언제 생기나? 빌린 커넥션을 닫지 않을 때. JdbcTemplate이나 try-with-resources를 쓰면 프레임워크가 반납을 보장한다.

참고: HikariCP (github.com/brettwooldridge/HikariCP) · Spring Boot Data Access 문서 · mysql2 Pool 옵션 · 실습 저장소 github.com/seoktae-lee/umc-backend-study

3. 두 프레임워크의 차이

🌱 Spring Boot — 이미 준비된 풀옵션

`application.yml` 의 DB 정보만 있으면, 서버가 켜질 때 HikariCP(기본 커넥션 풀)와 이를 쉽게 쓰는 JdbcTemplate을 스프링이 알아서 만들어 컨테이너에 넣어 둔다. **별도 설정 코드가 없다**.

```
# application.yml - 이게 전부
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/book_rental
    username: ${DB_USER:root}
    password: ${DB_PASSWORD:}
```

```
// Repository에서는 주입만 받아 쓴다
@Repository @RequiredArgsConstructor
public class BookRepository {
    private final JdbcTemplate jdbcTemplate; // ← 끝
}
```

🐱 NestJS — 직접 조립하는 Provider

```
// src/database.provider.ts
mysql.createPool({
  host: ..., user: ..., password: ..., database: ...,
  waitForConnections: true, // 꽉 차면 여러 대신 대기
  connectionLimit: 10, queueLimit: 0,
});
```

가볍고 모듈화된 프레임워크라, ORM 없이 순수 드라이버를 쓸 때는 풀을 **직접 선언** 하고 `app.module.ts` 에 등록해야 한다. (4주차 ORM 도입 시 사라지는 임시 코드)

5. 주의할 점

커넥션 누수(leak)

빌린 커넥션을 반납하지 않으면 풀이 말라 서버 전체가 멈춘다.

JdbcTemplate 이 빌리고 반납까지 처리하므로, Repository에서 커넥션을 직접 열고 닫는 코드를 쓰지 않는 것이 최선의 방법이다.

풀 크기는 클수록 좋은가 → 아니다

커넥션 하나당 DB 쪽에도 스레드·메모리가 붙는다. 앱 서버 여러 대가 각자 큰 풀을 잡으면 DB의 `max_connections` 를 넘겨 오히려 죽는다. HikariCP 문서도 "작게 유지하라"가 권장. 기본 10에서 시작해 측정 후 조정.